

UNIwersYTET RZESZOWSKI
WYDZIAŁ NAUK ŚCISŁYCH I TECHNICZNYCH
INSTYTUT INFORMATYKI



Oleksii Nawrocki
oa131400

Informatyka

Sprawozdanie z ćwiczeń laboratoryjnych z przedmiotu
Biometryczne Systemy Zabezpieczeń

Sprawozdania
Grupa: 02
Zespół nr: XX

Prowadzący
dr Zbigniew Gomółka

Rzeszów 2025

Spis treści

1. Laboratorium 1	7
1.1. Cel ćwiczenia	7
1.2. Wstęp teoretyczny	7
1.3. Opis wykorzystywanego oprogramowania i narzędzi	7
1.4. Zadania do wykonania samodzielnego	8
1.4.1. Zadanie 1	8
1.4.2. Zadanie 2	11
1.4.3. Zadanie 3	13
1.4.4. Zadanie 4	15
1.4.5. Zadanie 5	17
1.5. Wnioski	18
2. Laboratorium 2	20
2.1. Cel ćwiczenia	20
2.2. Wstęp teoretyczny	20
2.3. Opis wykorzystywanego oprogramowania i narzędzi	21
2.4. Zadania do wykonania samodzielnego	22
2.4.1. Zadanie 1	22
2.5. Wnioski	26
3. Laboratorium 3	27
3.1. Cel ćwiczenia	27
3.2. Wstęp teoretyczny	27
3.2.1. Szkieletyzacja obrazów binarnych	27
3.2.2. Algorytm Chin-Van-Stover-Iverson	28
3.2.3. Szkieletyzacja obrazów tonowanych	28
3.3. Zadania do wykonania samodzielnego	29
3.3.1. Zadanie 1	29
3.4. Wnioski	32
4. Laboratorium 4	34
4.1. Cel ćwiczenia	34
4.2. Wstęp teoretyczny	34
4.3. Opis wykorzystywanego oprogramowania i narzędzi	34
4.4. Zadania do wykonania samodzielnego	34
4.4.1. Zadanie X	34
4.5. Wnioski	34
Bibliografia	35
Spis rysunków	36

Spis tabel	37
Spis listingów	38
Oświadczenie studenta o samodzielności pracy	39

1. Laboratorium 1

Biometryczne Systemy Zabezpieczeń Ćwiczenia Laboratoryjne	
Temat	WPROWADZENIE DO BIBLIOTEKI MATLAB IMAGE PROCESSING ORAZ HISTOGRAMOWE METODY SEGMENTACJI OBRAZÓW.
Ćwiczenie nr:	01
Autorzy:	Oleksii Nawrocki
Grupa laboratoryjna	02
Zespół nr.	XX
Data wykonania ćwiczenia	23.04.2025
Data oddania ćwiczenia	13.05.2025

1.1. Cel ćwiczenia

Celem laboratorium jest zapoznanie się z podstawowymi funkcjami biblioteki Image Processing Toolbox w MATLAB-ie oraz zastosowanie metod segmentacji obrazów opartych na analizie histogramu. Należy zapoznać się z materiałami i przykładami zawartymi w niniejszych materiałach oraz zaproponować rozwiązania dla zadań do samodzielnego rozwiązania. Rozwiązania należy umieścić w sprawozdaniu, zgodnie z wymaganiami.

1.2. Wstęp teoretyczny

Histogram obrazu to funkcja przedstawiająca rozkład wartości pikseli w obrazie. Może być wykorzystywany do:

- Poprawy kontrastu obrazu (rozciąganie histogramu, wyrównywanie histogramu),
- Segmentacji obrazu na podstawie progów jasności (np. metoda Otsu),
- Analizy jakości obrazu, w tym identyfikacji prześwieconych lub niedoświetlonych obszarów.

Histogram w obrazie monochromatycznym można obliczyć za pomocą funkcji $h(k)$, określającej liczbę pikseli o poziomie jasności k :

$$h(k) = \sum_{(x,y) \in \Omega} \delta(I(x,y) - k), \quad (1.1)$$

gdzie δ to funkcja Kroneckera, a Ω oznacza zbiór pikseli w obrazie.

1.3. Opis wykorzystywanego oprogramowania i narzędzi

Do przeprowadzenia eksperymentu wykorzystano środowisko MATLAB oraz bibliotekę Image Processing Toolbox. Kluczowe funkcje wykorzystane w analizie to:

- `imhist` - obliczanie histogramu obrazu,
- `imadjust` - rozciąganie histogramu,

- `histeq` - wyrównywanie histogramu,
- `graythresh` oraz `imbinarize` - segmentacja metodą Otsu.

1.4. Zadania do wykonania samodzielnego

1.4.1. Zadanie 1

1.4.1.1. Treść

Konwersja obrazu na różne formaty przy użyciu MATLAB i Image Processing Toolbox:

Użyj funkcji dostępnych w bibliotece IPT w MATLAB-ie do przekształcenia obrazu. Na Rys. 9 przedstawiono schemat przepływu i konwersji różnych typów obrazów w MATLAB-ie, z wykorzystaniem funkcji dostępnych w IPT.

1.4.1.2. Pytania do analizy

- Jak zmienia się zakres wartości pikseli w kolejnych przekształceniach?
 - RGB: 3 kanały (R, G, B), każdy z zakresem [0, 255]
 - Monochromatyczny (gray): pojedynczy kanał, zakres [0, 255]
 - Binarny: dwa poziomy – 0 (czarny) i 1 (biały)
 - Indeksowany: każdy piksel to indeks w tablicy kolorów (zazwyczaj 0–255 przy 256 kolorach)
- Jaka jest różnica w zajmowanej pamięci między różnymi formatami obrazów?
 - **RGB**: największy rozmiar – 3 bajty na piksel (8 bitów × 3 kanały).
 - **Gray**: 1 bajt na piksel.
 - **Binarny**: 1 bit na piksel (oszczędność miejsca).
 - **Indeksowany**: 1 bajt na piksel + osobna tablica kolorów (colormap).
- Jakie mogą być zastosowania obrazów indeksowanych i binarnych w praktyce?
 - Indeksowane:
 - * kompresja obrazów do GIF.
 - * aplikacje z ograniczoną paletą barw (np. grafika retro, przeglądarki).
 - Binarne:
 - * analiza obiektów (segmentacja, maski).
 - * OCR (rozpoznawanie tekstu).
 - * wykrywanie krawędzi, konturów.

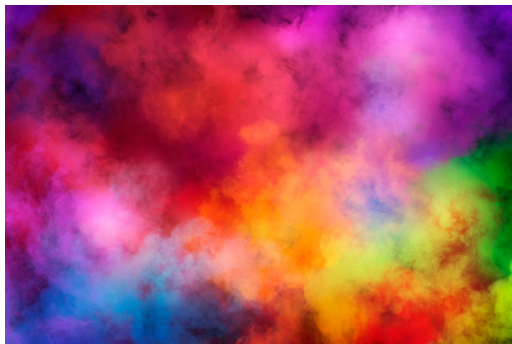
1.4.1.3. Przykładowy program

Listing 1.1. Wczytanie obrazu i stworzenie konwersji

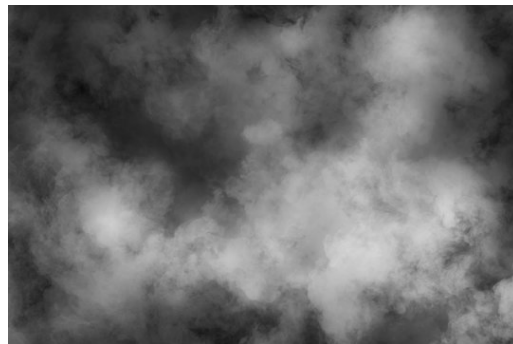
```
1 % Wczytaj obraz RGB
2 rgbImage = imread('colors.jpg'); % użyj dowolnego obrazu RGB
3 imshow(rgbImage);
4 title('Obraz oryginalny RGB');
```

```
5
6 % 1. Konwersja do obrazu monochromatycznego (odcienie szarości)
7 grayImage = rgb2gray(rgbImage);
8 figure, imshow(grayImage);
9 title('Obraz monochromatyczny');
10
11 % 2. Binaryzacja obrazu (konwersja do obrazu binarnego)
12 binaryImage = imbinarize(grayImage);
13 figure, imshow(binaryImage);
14 title('Obraz binarny');
15
16 % 3. Konwersja do obrazu indeksowanego (indeks + mapa kolorów)
17 [indexedImage, colormap] = rgb2ind(rgbImage, 256); % 256 kolorów
18 figure, imshow(indexedImage, colormap);
19 title('Obraz indeksowany');
20
21 % 4. Wyodrębnienie kanałów RGB
22 redChannel = rgbImage(:, :, 1);
23 greenChannel = rgbImage(:, :, 2);
24 blueChannel = rgbImage(:, :, 3);
25
26 figure,
27 subplot(1,3,1), imshow(redChannel), title('Kanał czerwony');
28 subplot(1,3,2), imshow(greenChannel), title('Kanał zielony');
29 subplot(1,3,3), imshow(blueChannel), title('Kanał niebieski');
30
31 % 5. Zapis do plików
32 imwrite(rgbImage, 'zadanielimages/obraz_rgb.png');
33 imwrite(grayImage, 'zadanielimages/obraz_szary.png');
34 imwrite(binaryImage, 'zadanielimages/obraz_binarny.png');
35 imwrite(indexedImage, colormap, 'zadanielimages/obraz_indeksowany.png');
36
37 % Zapis kanałów RGB
38 imwrite(redChannel, 'zadanielimages/kanal_czerwony.png');
39 imwrite(greenChannel, 'zadanielimages/kanal_zielony.png');
40 imwrite(blueChannel, 'zadanielimages/kanal_niebieski.png');
```

1.4.1.4. Obrazy



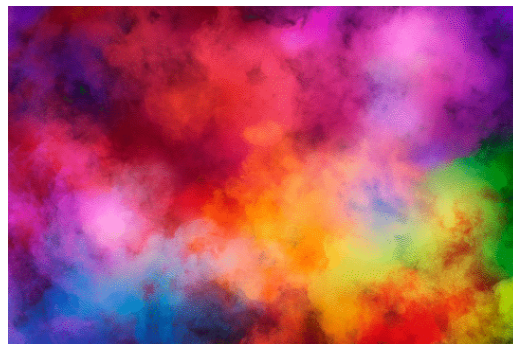
(a) Obraz RGB



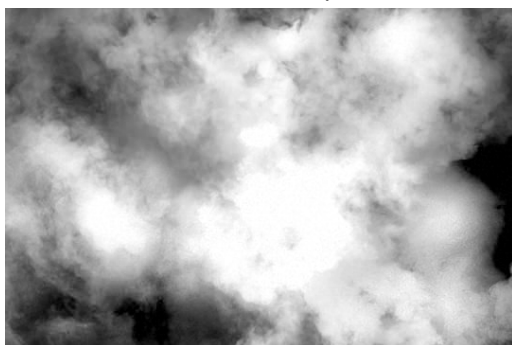
(b) Obraz szary



(c) Obraz binarny



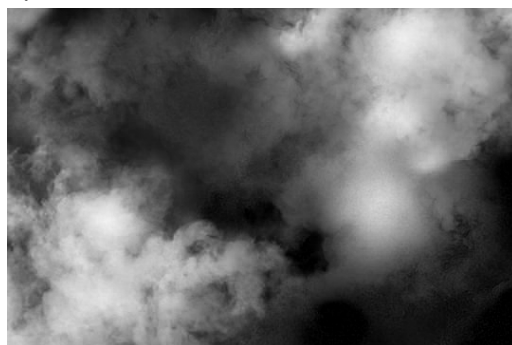
(d) Obraz indeksowany



(e) Kanał czerwony



(f) Kanał zielony



(g) Kanał niebieski

Rys. 1.1. Wizualizacja obrazów w różnych formatach i kanałach.

1.4.2. Zadanie 2

1.4.2.1. Treść

Zadanie 2. Histogram i operacje na histogramie.

W Matlab zaproponuj program, który pozwoli na:

- Wczytanie obrazu w skali szarości (`imread` → `rgb2gray` jeśli potrzeba).
- Obliczenie i wyświetlenie histogramu obrazu (`imhist`).
- Wykonaj rozciąganie histogramu (`imadjust` + `stretchlim`) i porównaj obraz przed i po operacji.
- Wykonaj wyrównanie histogramu (`histeq`) i porównaj wynik z oryginalnym obrazem oraz z histogramem rozciągniętym.
- Wyświetl histogramy wszystkich przekształconych obrazów.

1.4.2.2. Przykładowy program

Listing 1.2. Wczytanie obrazu i przetwarzanie obrazu histogramami

```
1 % Wczytanie obrazu (kolorowego lub w skali szarości)
2 rgbImage = imread('cameraman.jpg');
3 grayImage = rgb2gray(rgbImage); % Konwersja do skali szarości
4
5 % Wyświetlenie oryginalnego obrazu i jego histogramu
6 figure;
7 subplot(2,2,1), imshow(grayImage), title('Oryginalny obraz (gray)');
8 subplot(2,2,2), imhist(grayImage), title('Histogram oryginalny');
9
10 % Rozciąganie histogramu (stretching)
11 stretchedImage = imadjust(grayImage, stretchlim(grayImage));
12 subplot(2,2,3), imshow(stretchedImage), title('Po rozciąganiu histogramu');
13 subplot(2,2,4), imhist(stretchedImage), title('Histogram po rozciąganiu');
14
15 % Wyrównanie histogramu (equalization)
16 equalizedImage = histeq(grayImage);
17
18 % Nowa figura do porównań
19 figure;
20 subplot(2,2,1), imshow(equalizedImage), title('Po wyrównaniu histogramu');
21 subplot(2,2,2), imhist(equalizedImage), title('Histogram po wyrównaniu');
22 subplot(2,2,3), imshowpair(stretchedImage, equalizedImage, 'montage');
23 title('Porównanie: Rozciągnięty vs Wyrównany');
24
25 % Zapis obrazów (opcjonalnie)
26 imwrite(grayImage, 'zadanie2images/obraz_szary.png');
27 imwrite(stretchedImage, 'zadanie2images/obraz_stretched.png');
28 imwrite(equalizedImage, 'zadanie2images/obraz_equalized.png');
```



(a) Obraz wyrównany



(b) Obraz rozciągnięty

Rys. 1.2. Wizualizacja rozciągania i wyrównywania obrazów**1.4.2.3. Obrazy****1.4.2.4. Pytania do analizy**

- Jaka jest różnica między rozciąganiem a wyrównywaniem histogramu?
 - Rozciąganie (stretching): zmienia zakres jasności, przekształcając piksele tak, by rozciągnąć istniejący histogram do pełnego zakresu (np. 0–255).
 - Wyrównywanie (equalization): zmienia rozkład jasności, aby histogram był możliwie równomierny – poprawiając kontrast globalnie.
- Która metoda daje lepsze wyniki poprawy kontrastu?
 - Dla obrazów o niskim kontraście — wyrównanie często działa lepiej.
 - Dla obrazów z dobrze rozłożonymi poziomami jasności — rozciąganie może być wystarczające i mniej inwazyjne.
- Jakie są potencjalne zastosowania tych technik?
 - Poprawa jakości obrazów w:
 - * medycynie (np. obrazowanie rentgenowskie),
 - * nadzorze wizyjnym (monitoring),
 - * przetwarzaniu zdjęć satelitarnych,
 - * OCR (rozpoznawanie tekstu),
 - * automatycznej analizie obrazów przemysłowych.

1.4.3. Zadanie 3

1.4.3.1. Treść

Zadanie 3. Segmentacja obrazu na podstawie histogramu.

W Matlab zaproponuj program, który pozwoli na:

- Wczytanie obrazu w skali szarości.
- Obliczenie histogramu obrazu (imhist) i znalezienie potencjalnego progu segmentacji.
- Przeprowadź ręczną segmentację obrazu, stosując różne wartości progowe ($BW = I > \text{threshold}$).
- Zastosuj automatyczną segmentację metodą Otsu (graythresh + imbinarize).
- Porównaj efekty różnych progowań, wyświetlając wyniki obok siebie.

1.4.3.2. Przykładowy program

Listing 1.3. Wczytanie obrazu i segmentacja na podstawie histogramu

```
1 % Wczytanie obrazu i konwersja do skali szarości
2 rgbImage = imread('cameraman.jpg');
3 grayImage = rgb2gray(rgbImage);
4
5 % Wyświetlenie histogramu
6 figure;
7 imhist(grayImage);
8 title('Histogram obrazu w skali szarości');
9
10 % Segmentacja ręczna przy różnych progach
11 threshold1 = 80;
12 threshold2 = 120;
13 threshold3 = 160;
14
15 BW1 = grayImage > threshold1;
16 BW2 = grayImage > threshold2;
17 BW3 = grayImage > threshold3;
18
19 % Segmentacja automatyczna metodą Otsu
20 otsuThreshold = graythresh(grayImage); % zwraca wartość w zakresie [0, 1]
21 BW_otсу = imbinarize(grayImage, otsuThreshold);
22
23 % Wyświetlenie wyników
24 figure;
25 subplot(2,3,1), imshow(grayImage), title('Oryginalny obraz');
26 subplot(2,3,2), imshow(BW1), title(['Próg ręczny = ' num2str(threshold1)]);
27 subplot(2,3,3), imshow(BW2), title(['Próg ręczny = ' num2str(threshold2)]);
28 subplot(2,3,4), imshow(BW3), title(['Próg ręczny = ' num2str(threshold3)]);
29 subplot(2,3,5), imshow(BW_otсу), title(['Otsu (próg = ' num2str(round(otsuThreshold*255))
    ') ']);
30
31 % Zapis wyników (opcjonalnie)
32 imwrite(BW1, 'zadanie3images/progowanie_80.png');
33 imwrite(BW2, 'zadanie3images/progowanie_120.png');
34 imwrite(BW3, 'zadanie3images/progowanie_160.png');
35 imwrite(BW_otсу, 'zadanie3images/segmentacja_otсу.png');
```

1.4.3.3. Obrazy



(a) Progowanie 80



(b) Progowanie 120



(c) Progowanie 160



(d) Segmentacja Otsu

Rys. 1.3. Operacje segmentowanie progowaniem oraz otsu.

1.4.3.4. Pytania do analizy

- Jak dobrać optymalny próg segmentacji dla różnych obrazów?
 - **Ręcznie:** na podstawie histogramu – szukamy wyraźnych "dolinek"(oddzielenie klas).
 - **Automatycznie:** np. metoda Otsu – optymalizuje podział na dwie klasy o minimalnej wariancji wewnątrzklasowej.
- Kiedy metoda Otsu jest skuteczniejsza niż segmentacja ręczna?
 - Gdy histogram jest **dwumodalny** (dwa główne zbiory pikseli – tło i obiekt).
 - Gdy chcemy **zautomatyzować** segmentację wielu obrazów bez ręcznego ustawiania progów.
 - Gdy nie mamy wiedzy o zawartości obrazu i chcemy uzyskać ogólnie dobrą separację.

- Jakie mogą być praktyczne zastosowania binaryzacji?
 - Detekcja i zliczanie obiektów (np. komórek w mikroskopii).
 - OCR (rozpoznawanie tekstu).
 - Analiza konturów i kształtów.
 - Przemysł: detekcja defektów, systemy wizyjne.
 - Monitoring: wykrywanie ruchu, segmentacja obiektów.

1.4.4. Zadanie 4

1.4.4.1. Treść

Zadania 4: Detekcja obiektów na obrazie binarnym.

W Matlab zaproponuj program, który pozwoli na:

- Wykonasz segmentację obrazu metodą Otsu i uzyskasz obraz binarny.
- Zidentyfikuj obiekty na obrazie binarnym za pomocą funkcji `bwlabel` lub `regionprops`.
- Wyznacz podstawowe właściwości wykrytych obiektów (np. powierzchnię, obwód).
- Narysuj prostokątne obramowania wokół wykrytych obiektów (`rectangle`).
- Zwizualizuj oryginalny obraz z nałożonymi obramowaniami.

1.4.4.2. Przykładowy program

Listing 1.4. Wczytanie obrazu i dokonanie detekcji obiektów na obrazie

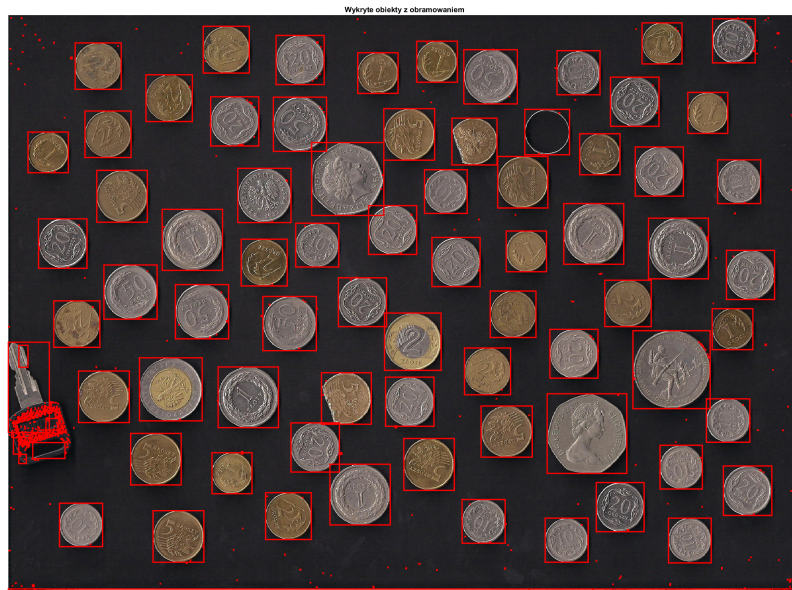
```
1 % Wczytanie obrazu i konwersja do skali szarości
2 rgbImage = imread('monety1.jpg'); % przykład dobrze kontrastowego obrazu
3 grayImage = rgb2gray(rgbImage);
4
5 % Segmentacja metodą Otsu
6 binaryImage = imbinarize(grayImage, graythresh(grayImage));
7
8 % Wypełnienie dziur (opcjonalne)
9 binaryImage = imfill(binaryImage, 'holes');
10
11 % Etykietowanie obiektów
12 labeledImage = bwlabel(binaryImage);
13
14 % Właściwości obiektów
15 props = regionprops(labeledImage, 'BoundingBox', 'Area', 'Perimeter');
16
17 % Wyświetlenie obrazu z nałożonymi obramowaniami
18 figure;
19 imshow(rgbImage);
20 title('Wykryte obiekty z obramowaniem');
21 hold on;
22
23 % Iteracja przez obiekty i rysowanie obramowań
24 for k = 1:length(props)
25     bbox = props(k).BoundingBox;
26     rectangle('Position', bbox, 'EdgeColor', 'r', 'LineWidth', 2);
```

```

27
28 % Wyświetlanie powierzchni i obwodu w konsoli (opcjonalnie)
29 fprintf('Obiekt %d: Powierzchnia = %.2f, Obwód = %.2f\n', k, props(k).Area, props(k)
        .Perimeter);
30 end
31
32 hold off;
33 exportgraphics(gca, 'zadanie4images/obiekty_z_obramowaniem.png');

```

1.4.4.3. Obrazy



Rys. 1.4. Detekcja obiektów na obrazie.

1.4.4.4. Pytania do analizy

1. Jakie problemy mogą wystąpić przy detekcji obiektów w zależności od jakości segmentacji?

- Zbyt niska jakość segmentacji może prowadzić do:
 - scalania sąsiadujących obiektów w jeden,
 - pocięcia jednego obiektu na kilka części,
 - pomijania obiektów o słabym kontraście,
 - szumu – detekcji fałszywych obiektów.

2. Jak można poprawić dokładność detekcji?

- Filtracja obrazu przed segmentacją (np. `medfilt2`, `imgaussfilt`).
- Morfologia: `imopen`, `imclose`, `bwareaopen` – usuwa drobne obiekty/szumy.
- Dostosowanie progu segmentacji ręcznie lub przez adaptacyjną binaryzację (`adaptthresh`).
- Użycie maski ROI lub filtrowanie po właściwościach (Area, Eccentricity, itp.).

3. Jakie są praktyczne zastosowania tej metody?

- **Biomedycyna:** zliczanie komórek, wykrywanie struktur w mikroskopii.
- **Przemysł:** kontrola jakości (np. wykrywanie wad, pomiar kształtu).

- **Robotyka/Widzenie maszynowe:** identyfikacja i lokalizacja obiektów.
- **Bezpieczeństwo:** wykrywanie tablic rejestracyjnych, ludzi, ruchu.

1.4.5. Zadanie 5

1.4.5.1. Treść

Zadanie 5. Segmentacja wieloprogowa

W Matlab zaproponuj program, który pozwoli na:

- Wczytasz obraz w skali szarości.
- Obliczysz histogram obrazu i znajdziesz dwa optymalne progi metodą Otsu (multithresh).
- Utwórz obraz segmentowany na trzy klasy (imquantize).
- Zwizualizuj wynik segmentacji z użyciem różnych kolorów (label2rgb).
- Porównaj segmentację wieloprogową z binaryzacją jednokryterialną.

1.4.5.2. Przykładowy program

Listing 1.5. Wczytanie obrazu i dokonanie detekcji obiektów na obrazie

```
1 % Wczytanie obrazu i konwersja do skali szarości
2 rgbImage = imread('cameraman.jpg');
3 grayImage = rgb2gray(rgbImage);
4
5 % Wyznaczenie 2 progów metodą Otsu (wieloprogowość)
6 thresholds = multithresh(grayImage, 2); % [t1, t2]
7
8 % Segmentacja obrazu na 3 klasy
9 segmentedImage = imquantize(grayImage, thresholds);
10
11 % Kolorowanie klas dla wizualizacji
12 rgbSegmented = label2rgb(segmentedImage, 'jet', 'k');
13
14 % Binarizacja klasyczna dla porównania
15 binaryImage = imbinarize(grayImage, graythresh(grayImage));
16
17 % Wyświetlenie wyników
18 figure;
19 subplot(2,2,1), imshow(grayImage), title('Oryginalny obraz (gray)');
20 subplot(2,2,2), imshow(binaryImage), title('Binarizacja jednokryterialna');
21 subplot(2,2,3), imshow(segmentedImage, []), title('Obraz segmentowany (3 klasy)');
22 subplot(2,2,4), imshow(rgbSegmented), title('Segmentacja wieloprogowa (kolor)');
23
24 % Zapis obrazów (opcjonalny)
25 imwrite(binaryImage, 'zadanie5images/binaryzacja_otsu.png');
26 imwrite(rgbSegmented, 'zadanie5images/segmentacja_wieloprogowa.png');
```

1.4.5.3. Obrazy



(a) Progowanie 80



(b) Progowanie 120

Rys. 1.5. Operacje segmentowanie wieloprogowego.

1.4.5.4. Pytania do analizy

1. W jakich przypadkach segmentacja wieloprogowa daje lepsze wyniki niż binaryzacja?
 - Gdy obraz zawiera **więcej niż dwa poziomy jasności** reprezentujące różne obiekty lub tło (np. tło, cień, obiekt).
 - Przy obrazach z **wieloma klasami intensywności** – np. zdjęcia medyczne, zdjęcia z mikroskopu, obrazy satelitarne.
2. Jakie są ograniczenia segmentacji wieloprogowej?
 - **Wymaga wiedzy** o liczbie klas (ile progów wyznaczyć).
 - **Czasochłonna** przy większej liczbie progów lub dużych obrazach.
 - Czasem nie sprawdza się przy słabym kontraście lub zaszumionym obrazie – klasy mogą się zlewać.
3. Jak można poprawić jakość segmentacji?
 - **Filtracja** przed segmentacją (imgaussfilt, medfilt2).
 - **Użycie morfologii** po segmentacji (imopen, imclose, bwareaopen).
 - Dobór progów ręcznie lub przez adaptacyjne metody (adaptthresh).
 - W połączeniu z analizą cech (regionprops) można odfiltrować obiekty niepasujące do oczekiwań.

1.5. Wnioski

Na podstawie przeprowadzonych eksperymentów można stwierdzić, że:

- Histogram jest przydatnym narzędziem w analizie jakości obrazu biometrycznego.

- Wyrównanie histogramu poprawia kontrast i może zwiększać czytelność obrazu.
- Segmentacja oparta na histogramie, w szczególności metoda Otsu, pozwala na skuteczne wydzielenie istotnych cech obrazu.

2. Laboratorium 2

Biometryczne Systemy Zabezpieczeń	
Ćwiczenia Laboratoryjne	
Temat	DETEKCJA KRAWĘDZI W OBRAZIE METODAMI SPLOTOWYMI.
Ćwiczenie nr:	02
Autorzy:	Oleksii Nawrocki
Grupa laboratoryjna	02
Zespół nr.	XX
Data wykonania ćwiczenia	26.04.2025
Data oddania ćwiczenia	13.05.2025

2.1. Cel ćwiczenia

Filtracja splotowa to jedno z podstawowych narzędzi przetwarzania obrazów, wykorzystywane do wygładzania, wykrywania krawędzi, redukcji szumów oraz innych operacji poprawiających jakość obrazu. Splot (ang. convolution) polega na przekształceniu obrazu poprzez nałożenie na niego maski (jądra) filtrującego. Celem laboratorium jest implementacja algorytmu splotowego w MATLAB-ie oraz zbadanie wpływu różnych filtrów na obraz.

2.2. Wstęp teoretyczny

2.2.0.1. Filtracja splotowa

Filtracja splotowa (ang. convolution filtering) jest podstawową metodą przetwarzania obrazów w dziedzinie przestrzennej. Polega ona na nakładaniu na obraz niewielkiej maski (kernela) i obliczaniu nowych wartości pikseli jako sumy ważonej sąsiednich pikseli zgodnie z wartościami w masce. W praktyce oznacza to, że dla każdego punktu obrazu uwzględnia się przyległe piksele, których wartości mnoży się przez odpowiadające wagi z maski, a następnie sumuje. Taki proces pozwala uzyskać różne efekty – od wygładzania (rozmycia) poprzez wyostanie po detekcję krawędzi, w zależności od przyjętych wag w masce filtra.

2.2.0.2. Filtry krawędziowe (Roberts, Prewitt, Sobel, Laplace)

Operator Roberta – jest jednym z pierwszych detektorów krawędzi wykorzystujących maski różnicowe. Używa dwóch małych masek 2×2 do przybliżenia gradientu obrazu w kierunkach ukośnych. Filtr ten jest bardzo prosty obliczeniowo (maski zawierają tylko liczby całkowite), jednak jego wadą jest duża wrażliwość na szum. Wynikiem działania operatora Roberta są zmiany intensywności wzdłuż przekątnych, co prowadzi do wykrywania krawędzi o orientacji diagonalnej.

Operator Prewitta – wykorzystuje dwie maski 3×3 do przybliżenia pochodnych obrazu w kierunku poziomym i pionowym. Każda z masek jest wynikiem splotu filtru różnicującego z filtrem uśredniającym. Dzięki temu operator Prewitta częściowo wygładza obraz, co poprawia wykrywanie krawędzi poziomych i pionowych. Generowany wektor gradientu obrazu wskazuje siłę krawędzi (długość wektora) oraz jej orientację (kierunek wektora).

Operator Sobela – podobnie jak Prewitt, używa masek 3×3 , jednak nadaje większe wagi środkowym elementom masek (wartość 2), co skutkuje silniejszym wygładzeniem i zwiększoną odpornością na szum. Operator Sobela wydajnie przybliża gradient intensywności obrazu, szczególnie eksponując miejsca gwałtownych zmian jasności.

Operator Laplace’a – oparty jest na drugiej pochodnej obrazu (Laplasjanie) i wykrywa zmiany intensywności niezależnie od kierunku. Jest to filtr drugiego rzędu, izotropowy w przestrzeni 2D. Operator Laplace’a uwypukla obszary szybkiej zmiany jasności, ale jest również podatny na szumy. Charakterystyczną cechą wyników Laplasjanu jest wykrywanie miejsc przecięcia zera (zero-crossings), które odpowiadają lokalizacjom krawędzi.

2.2.0.3. Filtr medianowy a filtr uśredniający

Filtr medianowy jest nieliniowym filtrem, w którym wartość piksela zastępowana jest medianą wartości z sąsiadujących pikseli. Dzięki temu skutecznie usuwa szum impulsowy typu „sól i pieprz”, zachowując jednocześnie ostrość krawędzi. W przeciwieństwie do filtrów liniowych, filtr medianowy nie wprowadza nowych wartości spoza przedziału istniejących intensywności, co zapobiega powstawaniu artefaktów.

Filtr uśredniający (średnia arytmetyczna) jest filtrem liniowym, który zastępuje wartość piksela średnią z jego sąsiadów. Filtr ten skutecznie redukuje szumy o rozkładzie Gaussa, jednak prowadzi do rozmycia krawędzi i utraty szczegółów. W praktyce filtr medianowy jest preferowany do usuwania szumu impulsowego, natomiast filtr uśredniający stosuje się głównie do ogólnego wygładzania obrazu.

2.2.0.4. Szum typu „sól i pieprz” (impulsowy)

Szum typu „sól i pieprz” to rodzaj szumu impulsowego, w którym losowe piksele przyjmują wartości minimalne (0, „pieprz”) lub maksymalne (255, „sól”). Zazwyczaj występuje na skutek błędów transmisji, uszkodzeń matrycy detektora lub błędów zapisu danych. Do jego usunięcia najlepiej nadają się filtry medianowe, które skutecznie eliminują pojedyncze anomalie bez znaczącej utraty ostrości obrazu. Filtry liniowe, takie jak filtr uśredniający, są mniej skuteczne, ponieważ rozmywają szum na większe obszary obrazu.

2.3. Opis wykorzystywanego oprogramowania i narzędzi

Do przeprowadzenia eksperymentu wykorzystano środowisko MATLAB oraz bibliotekę Image Processing Toolbox. Kluczowe funkcje używane w analizie to:

- `imfilter()` – funkcja MATLAB służąca do filtracji obrazów za pomocą splotu lub korelacji z podaną maską. Pozwala na wybór sposobu obramowania granic obrazu oraz zachowuje typ danych wejściowych. Funkcja `imfilter()` umożliwia wygodne operacje na obrazach, zachowując ich oryginalne rozmiary.
- `conv2()` – funkcja realizująca dwuwymiarowy klasyczny splot, domyślnie zwracająca wynik o większym rozmiarze niż obraz wejściowy. Wynikowy obraz jest typu zmiennoprzecinkowego (double). `conv2()` jest wydajniejsza dla dużych masek, jednak wymaga ręcznego dostosowania rozmiaru i rzutowania typu wynikowego.

2.4. Zadania do wykonania samodzielnego

2.4.1. Zadanie 1

2.4.1.1. Treść

Filtrowanie splotowe obrazu oka:

Wykorzystując dowolny obraz biometryczny tęczówki oka wykonaj program dokonujący filtracji splotowej filtrami Robertsa, Prewitta, Sobela oraz Laplace'a. Uwzględnij w programie możliwość zadawania rozmiaru maski kernela 3x3 oraz 5x5.

- Wyświetl wyniki filtracji w oknie wykorzystując funkcję subplot.
- Porównaj efekty filtracji dla różnych filtrów. Sprawdź wpływ zmiany rozmiaru maski filtra na wynik końcowy.
- Wprowadź do obrazu zadaną porcję szumu SaltAndPepper wykorzystując funkcję imnoise.
- Przeprowadź na obrazie wejściowym filtrację medianową i uśredniającą a następnie porównaj otrzymane wyniki. Ponów procedurę wykrycia krawędzi w obrazie; przedstaw w sprawozdaniu dyskusję otrzymanych wyników.
- Wczytaj obraz kolorowy (peppers.png) i zastosuj filtrację na każdej składowej R, G, B osobno. Połącz prefiltrowane składowe w nowy obraz i wyświetl wynik.
- Sprawdź, jak działa imfilter(I, h, 'same') w porównaniu do conv2(). Czy wyniki są identyczne? Jakie są różnice?

2.4.1.2. Przykładowy program

Listing 2.1. Wczytanie obrazu i stworzenie konwersji

```

1 % Tworzenie katalogu, jeśli nie istnieje
2 outputFolder = 'zadanielimages';
3 if ~exist(outputFolder, 'dir')
4     mkdir(outputFolder);
5 end
6
7 % Wczytaj obraz biometryczny
8 I = imread('oko1.png');
9 I_gray = im2gray(I);
10 I_gray = im2double(I_gray);
11
12 % Definicje masek
13 roberts_mask = [0 0 0; -1 0 0; 0 1 0];
14 prewitt_mask = [-1 -1 -1; 0 0 0; 1 1 1];
15 sobel_mask = [-1 -2 -1; 0 0 0; 1 2 1];
16 laplace_mask1 = [0 -1 0; -1 4 -1; 0 -1 0];
17 laplace_mask2 = [-1 -2 -1; -2 4 -2; -1 -2 -1];
18
19 %% Filtracja 3x3
20 figure;
21 subplot(2,3,1), imshow(I_gray), title('Oryginał');
22 subplot(2,3,2), imshow(imfilter(I_gray, roberts_mask, 'replicate')), title('Roberts');
```

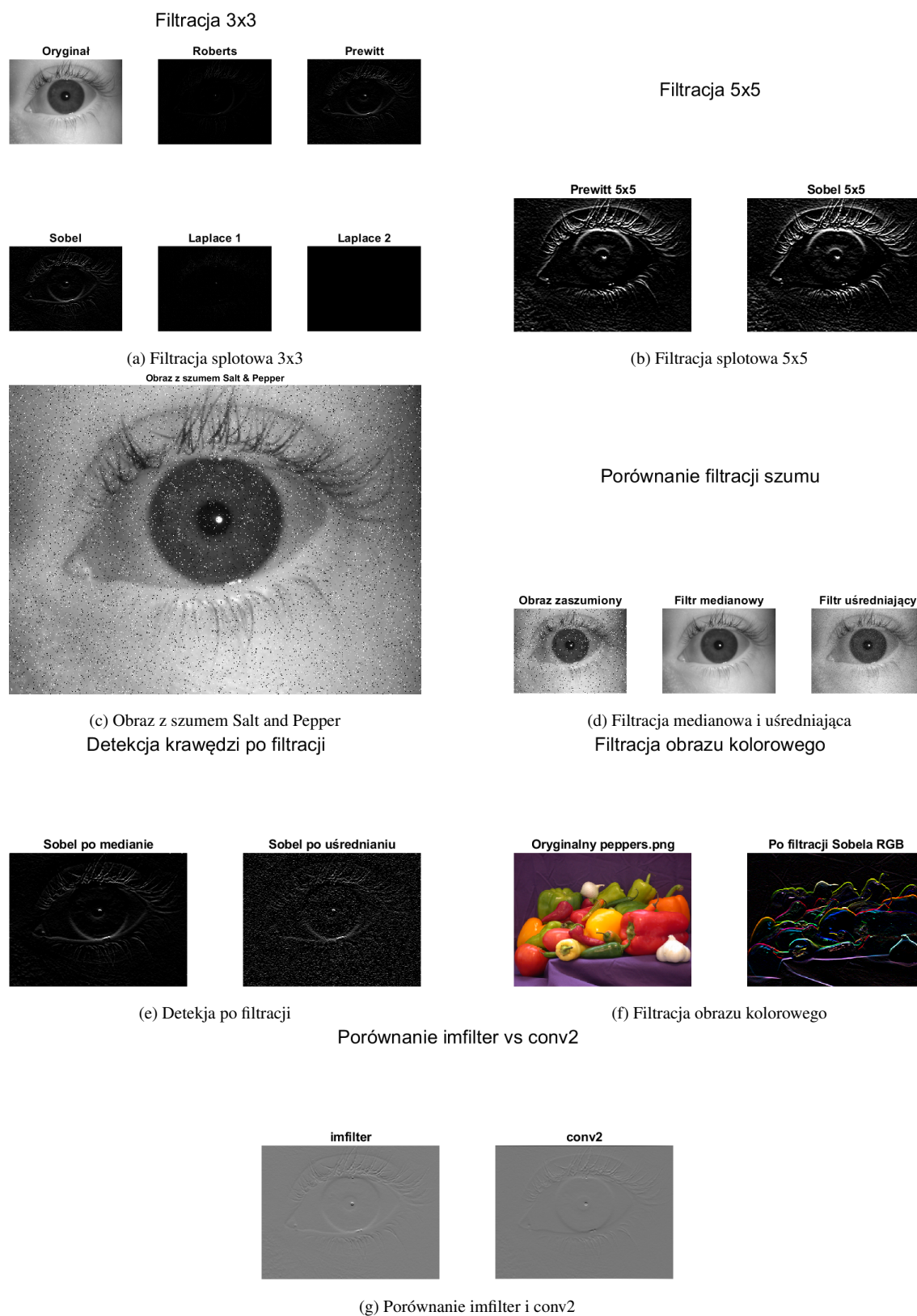
```

23 subplot(2,3,3), imshow(imfilter(I_gray, prewitt_mask, 'replicate')), title('Prewitt');
24 subplot(2,3,4), imshow(imfilter(I_gray, sobel_mask, 'replicate')), title('Sobel');
25 subplot(2,3,5), imshow(imfilter(I_gray, laplace_mask1, 'replicate')), title('Laplace 1')
    ;
26 subplot(2,3,6), imshow(imfilter(I_gray, laplace_mask2, 'replicate')), title('Laplace 2')
    ;
27 sgtitle('Filtracja 3x3');
28
29 exportgraphics(gcf, fullfile(outputFolder, 'filtracja_3x3.png'));
30
31 %% Filtracja 5x5
32 hprewitt5 = imresize(fspecial('prewitt'), [5 5], 'nearest');
33 hsobel5 = imresize(fspecial('sobel'), [5 5], 'nearest');
34
35 figure;
36 subplot(1,2,1), imshow(imfilter(I_gray, hprewitt5, 'replicate')), title('Prewitt 5x5');
37 subplot(1,2,2), imshow(imfilter(I_gray, hsobel5, 'replicate')), title('Sobel 5x5');
38 sgtitle('Filtracja 5x5');
39
40 exportgraphics(gcf, fullfile(outputFolder, 'filtracja_5x5.png'));
41
42 %% Dodanie szumu Salt & Pepper
43 noisy = imnoise(I_gray, 'salt & pepper', 0.05);
44
45 figure;
46 imshow(noisy);
47 title('Obraz z szumem Salt & Pepper');
48
49 exportgraphics(gcf, fullfile(outputFolder, 'szum_salt_pepper.png'));
50
51 %% Filtracja medianowa i uśredniająca
52 median_filtered = medfilt2(noisy, [3 3]);
53 average_filtered = imfilter(noisy, fspecial('average', [3 3]), 'replicate');
54
55 figure;
56 subplot(1,3,1), imshow(noisy), title('Obraz zaszumiony');
57 subplot(1,3,2), imshow(median_filtered), title('Filtr medianowy');
58 subplot(1,3,3), imshow(average_filtered), title('Filtr uśredniający');
59 sgtitle('Porównanie filtracji szumu');
60
61 exportgraphics(gcf, fullfile(outputFolder, 'filtracja_szumu.png'));
62
63 %% Detekcja krawędzi po filtracji
64 figure;
65 subplot(1,2,1), imshow(imfilter(median_filtered, sobel_mask, 'replicate')), title('Sobel
    po medianie');
66 subplot(1,2,2), imshow(imfilter(average_filtered, sobel_mask, 'replicate')), title('
    Sobel po uśrednianiu');
67 sgtitle('Detekcja krawędzi po filtracji');
68
69 exportgraphics(gcf, fullfile(outputFolder, 'detekcja_krawedzi_po_filtracji.png'));
70
71 %% Filtracja obrazu kolorowego peppers.png
72 RGB = imread('peppers.png');
73 R = RGB(:, :, 1);
74 G = RGB(:, :, 2);
75 B = RGB(:, :, 3);

```

```
76
77 Rf = imfilter(R, sobel_mask, 'replicate');
78 Gf = imfilter(G, sobel_mask, 'replicate');
79 Bf = imfilter(B, sobel_mask, 'replicate');
80
81 filtered_RGB = cat(3, Rf, Gf, Bf);
82
83 figure;
84 subplot(1,2,1), imshow(IMG), title('Oryginalny peppers.png');
85 subplot(1,2,2), imshow(filtered_RGB, []), title('Po filtracji Sobela RGB');
86 sgtitle('Filtracja obrazu kolorowego');
87
88 exportgraphics(gcf, fullfile(outputFolder, 'filtracja_rgb.png'));
89
90 %% Porównanie imfilter vs conv2
91 imf = imfilter(I_gray, sobel_mask, 'same', 'replicate');
92 convf = conv2(I_gray, sobel_mask, 'same');
93
94 figure;
95 subplot(1,2,1), imshow(imf, []), title('imfilter');
96 subplot(1,2,2), imshow(convf, []), title('conv2');
97 sgtitle('Porównanie imfilter vs conv2');
98
99 exportgraphics(gcf, fullfile(outputFolder, 'porownanie_imfilter_conv2.png'));
```

2.4.1.3. Obrazy



Rys. 2.1. Wizualizacja obrazów w różnych filtracjach splotowych.

2.5. Wnioski

Na podstawie przeprowadzonych eksperymentów można stwierdzić, że:

- **Filtry krawędziowe (Roberts, Prewitt, Sobel, Laplace)** skutecznie wykrywają zmiany intensywności w obrazie, jednak ich charakterystyka znacząco się różni. Operator Roberta jest prosty i szybki obliczeniowo, lecz wykazuje dużą wrażliwość na szum i słabo radzi sobie z krawędziami innymi niż ukośne. Operator Prewitta oraz Sobela są bardziej odpornymi filtrami, przy czym filtr Sobela oferuje lepsze wygładzenie dzięki większemu obciążeniu środkowych pikseli. Operator Laplace’a umożliwia wykrycie krawędzi niezależnie od kierunku, ale jego wrażliwość na szumy jest jeszcze większa niż filtrów pierwszorzędowych.
- **Rozmiar maski filtra** wpływa na wynik filtracji: maski 5x5 zapewniają mocniejsze wygładzanie i bardziej rozmyte krawędzie w porównaniu do masek 3x3. Zwiększenie rozmiaru maski powoduje, że filtr staje się mniej czuły na drobne detale i szum, ale za to mniej precyzyjnie lokalizuje krawędzie.
- **Wpływ szumu Salt&Pepper** na jakość obrazu jest znaczny. Obecność tego typu zakłóceń prowadzi do licznych, losowych zaburzeń w obrazie, które mogą zostać błędnie zinterpretowane jako krawędzie podczas procesu filtracji.
- **Filtr medianowy** wykazuje wysoką skuteczność w usuwaniu szumu typu Salt&Pepper przy zachowaniu krawędzi oraz struktury obrazu. W przeciwieństwie do niego, **filtr uśredniający** rozmywa obraz, prowadząc do rozmycia granic i mniejszej skuteczności w eliminacji impulsowych zakłóceń.
- **Filtracja obrazu kolorowego** (na osobnych kanałach RGB) umożliwia skuteczne przetwarzanie obrazów wielokanałowych. Jednak filtracja niezależnie każdego kanału może prowadzić do powstania artefaktów kolorystycznych, zwłaszcza w obszarach o silnych zmianach koloru.
- **Porównanie funkcji `imfilter` oraz `conv2`** wykazało, że obie metody dają podobne wyniki przy odpowiednim doborze opcji (parametr `'same'`). Funkcja `imfilter` jest bardziej elastyczna, oferując możliwość wyboru metody obsługi brzegów obrazu oraz zachowania typu danych. Z kolei `conv2` jest bardziej „surowa” i wymaga ręcznego dostosowania typu i rozmiaru wyniku.

Podsumowując, odpowiedni dobór metody filtracji, rozmiaru maski oraz sposobu redukcji szumów ma kluczowe znaczenie dla skutecznej analizy obrazów biometrycznych i przetwarzania obrazów cyfrowych w ogóle.

3. Laboratorium 3

Biometryczne Systemy Zabezpieczeń	
Ćwiczenia Laboratoryjne	
Temat	OPERACJE MORFOLOGICZNE NA OBRAZACH
Ćwiczenie nr:	03
Autorzy:	Oleksii Nawrocki
Grupa laboratoryjna	02
Zespół nr.	XX
Data wykonania ćwiczenia	26.04.2025
Data oddania ćwiczenia	13.05.2025

3.1. Cel ćwiczenia

Operacje morfologiczne są kluczowym narzędziem w przetwarzaniu obrazów, szczególnie w analizie kształtów i struktury obiektów. Dzięki wykorzystaniu odpowiednich elementów strukturalnych można dostosować te operacje do różnych zadań, takich jak segmentacja, poprawa jakości obrazu czy analiza konturów. Celem laboratorium jest zapoznanie się z operacjami morfologicznymi na obrazach.

3.2. Wstęp teoretyczny

Operacje morfologiczne to techniki przetwarzania obrazów stosowane głównie do analizy kształtu i struktury obiektów w obrazach binarnych oraz, w pewnym stopniu, w obrazach w skali szarości. Ich główną cechą jest wykorzystanie elementów strukturalnych do modyfikowania obrazu.

Szkieletyzacja to proces odnoszący się do ścieniania linii konturowych obiektu, który znalazł zastosowanie w wielu dziedzinach, takich jak:

- medycyna (analiza białych ciałek krwi, chromosomów, naczyń wieńcowych),
- kryminalistyka (rozpoznawanie odcisków palców),
- przemysł elektroniczny (analiza obwodów drukowanych),
- wektoryzacja obrazów rastrowych.

W niniejszym opracowaniu przedstawiono sposób działania wybranych algorytmów szkieletyzujących: Arcelli-Levialdi-Pavlidis, Chin-Van-Stover-Iverson oraz Rosenfeld-Peleg. Zaprojektowano także aplikację umożliwiającą użytkownikowi wprowadzanie własnych szablonów szkieletyzujących oraz obserwowanie otrzymanych wyników.

3.2.1. Szkieletyzacja obrazów binarnych

Pośród klasycznych metod ścieniania (*skeletoning* lub *thinning algorithms*), wyróżnić można trzy główne podejścia:

- **Wypełnianie działów wodnych** (dla obrazów w skali szarości),

- **Hit or Miss Transform** (dla obrazów binarnych i szarych),
- **Medial Axis Transform** (posiadająca przekształcenie odwrotne).

Podstawą tych metod są dwa główne przekształcenia morfologiczne: erozja i dylatacja. Formalnie, dla obrazów binarnych:

$$A \oplus B = \bigcup_{b \in B} \left(\bigcup_{a \in A} (a + b) \right) \quad (3.1)$$

$$A \ominus B = \bigcap_{b \in B} \left(\bigcup_{a \in A} (a + b) \right) \quad (3.2)$$

gdzie A to obraz, B to maska (element strukturalny), $\oplus$ oznacza dylatację, a $\ominus$ erozję. Znak $\bigcup$ oznacza sumę zbiorów, a $\bigcap$ ich część wspólną.

Hit or Miss Transform można opisać następująco:

- przykładamy element strukturalny b do każdego punktu obrazu,
- jeśli b pokrywa się z sąsiedztwem - zmieniamy wartość piksela (zwykle na 0),
- w przeciwnym wypadku piksel pozostaje niezmieniony.

Algorytmy takie jak Arcelli-Levialdi stosują szablony operujące na oknie bieżącego piksela oraz jego ośmiu sąsiadów. Możliwa jest także modyfikacja szablonów (np. użycie bitów typu "don't care") w celu uzyskania lepszego efektu szkieletyzacji gałęzi pionowych.

3.2.2. Algorytm Chin-Van-Stover-Iverson

Algorytm CVSI porównuje sąsiedztwo piksela z zestawem szablonów (1–8). Jeśli spełniony jest warunek zgodności z szablonem, następuje dodatkowe porównanie z szablonami 9 i 10.

- Jeśli otoczenie nie różni się od żadnego z szablonów — piksel pozostaje (jest częścią szkieletu).
- Jeśli różni się — piksel zostaje usunięty.

Aplikacja umożliwia ręczną edycję szablonów przez użytkownika (zmiana wartości 0, 1, x za pomocą kliknięcia).

3.2.3. Szkieletyzacja obrazów tonowanych

Naturalnym rozwinięciem szkieletyzacji obrazów binarnych jest szkieletyzacja obrazów w skali szarości. Przykładowe podejścia:

- erozja (Rosenfeld-Peleg),
- przekształcenie typu top-hat.

Ogólna translacja sygnału $f(x)$:

$$f(x) = f(x - z) \quad (3.3)$$

$$(f \oplus y)(x) = f(x) + y \quad (3.4)$$

Obie translacje można łączyć:

$$(f_z \oplus y)(x) = f(x - z) + y \quad (3.5)$$

Jeśli przyjmujemy, że $f(x) = -\infty$ poza obrazem, to dziedzinę obrazu określimy jako:

$$D[f] = \{x : f(x) > -\infty\} \quad (3.6)$$

Jeżeli $g(x) \leq f(x)$ dla każdego x , to:

$$D[g] \subseteq D[f] \quad (3.7)$$

Erozję definiujemy jako:

$$(f \wedge g)(x) = \begin{cases} \min\{f(x), g(x)\}, & x \in D[f] \cap D[g] \\ -\infty, & \text{inaczej} \end{cases} \quad (3.8)$$

Przy zastosowaniu płaskiego elementu strukturyzującego ($g_z(x) = 0$), erozję można zapisać jako:

$$(f \ominus g)(x) = \min\{f(x) - g_z(x)\} \quad (3.9)$$

co odpowiada filtracji typu *moving minimum*.

Do obserwacji zachowania algorytmu Rosenfeld-Peleg zaprojektowano obraz syntetyczny zawierający cylindryczną falę sinusoidalną.

3.3. Zadania do wykonania samodzielnego

3.3.1. Zadanie 1

3.3.1.1. Treść

1. Załaduj obraz testowy oraz przygotuj obraz do realizacji dalszych ćwiczeń o rozmiarach:
2. Wykonaj program realizujący algorytm ścieniania, którego zestaw kerneli zamieszczono poniżej:
3. Do zaprogramowania kolejnych szablonów i masek wykorzystaj poniższy fragment kodu źródłowego (funkcja `intbin8` formuje 8-bitową reprezentację liczby dziesiętnej przekazywanej parametrem `i`):
4. Sprawdź poprawność działania algorytmu na innym obrazie testowym, dla zmodyfikowanych parametrów filtracji morfologicznej jak poniżej:
5. Wyświetl wyniki otrzymanych filtracji wraz z podpisem w oknie wykorzystując funkcję `subplot`.
6. Wyjaśnij zjawisko zmieniającej się szybkości wykonywania algorytmu w zależności od fazy jego wykonywania.
7. Wykonaj wersję algorytmu realizującego szkieletyzację wg Chin-Wan-Stover-Iverson, przyjmując zestaw kerneli jak poniżej
8. Przeprowadź dyskusję otrzymanych wyników jak dla pkt 5-6.

3.3.1.2. Przykładowy program

Listing 3.1. Wczytanie obrazu i stworzenie konwersji

```

1 %% Zadanie - Ścienianie + Szkieletyzacja Chin-Wan-Stover-Iverson
2
3 clear; close all; clc;
4
5 %% 1. Wczytanie obrazu
6 [I, map] = imread('finger.jpg');
7 I = im2bw(I); % jeśli obraz nie jest jeszcze binarny
8
9 % Funkcja pomocnicza - konwersja liczby na 8-bitowy wektor binarny
10 intbin8 = @(i) bitget(uint8(i), 8:-1:1);
11
12 %% 2. Definicja masek (standardowe ścienianie)
13 s = [192 80 12 5 3 65 48 20];
14 m = [206 87 236 117 59 93 179 213];
15
16 %% 3. Funkcja thinning (standardowa)
17 function J = thinning(I, s, m)
18     J = I;
19     done = false;
20     while ~done
21         marker = false(size(J));
22         for k = 1:length(s)
23             % Przygotuj sąsiedztwo 8-nearest
24             N = circshift(J, [-1, 0]);
25             NE = circshift(J, [-1, 1]);
26             E = circshift(J, [0, 1]);
27             SE = circshift(J, [1, 1]);
28             S_ = circshift(J, [1, 0]);
29             SW = circshift(J, [1, -1]);
30             W = circshift(J, [0, -1]);
31             NW = circshift(J, [-1, -1]);
32
33             neighbors = cat(3, N, NE, E, SE, S_, SW, W, NW);
34
35             % Konwersja na 8-bitowe liczby
36             binImage = zeros(size(J), 'uint8');
37             for bit = 1:8
38                 binImage = bitor(binImage, uint8(neighbors(:, :, bit)) * 2^(8-bit));
39             end
40
41             % Warunek maski
42             cond = bitand(binImage, uint8(m(k))) == uint8(s(k));
43             marker = marker | (cond & J);
44         end
45         J_old = J;
46         J(marker) = 0;
47         done = isequal(J, J_old);
48     end
49 end
50
51 %% 4. Ścienianie - standardowe i zmodyfikowane parametry
52 J_standard = thinning(I, s, m);
53

```

```

54 % Zmodyfikowane parametry
55 s_mod = [192 2 12 5 3 65 48 20];
56 m_mod = [206 58 236 117 59 93 179 213];
57 J_modified = thinning(I, s_mod, m_mod);
58
59 %% 5. Szkieletyzacja Chin-Wan-Stover-Iverson
60
61 % Definicja kerneli według Twojej tabelki
62 % (przekształcenie na binarne maski)
63 s_cwsi = [0 1 1 0 0 1 0 0 1 1];
64 m_cwsi = [0 1 1 0 0 1 0 0 1 1];
65
66 % Funkcja szkieletyzacji
67 function J = cwsi_skeletonization(I)
68     J = I;
69     done = false;
70     while ~done
71         marker = false(size(J));
72
73         % Sąsiedztwo 8
74         N = circshift(J, [-1, 0]);
75         NE = circshift(J, [-1, 1]);
76         E = circshift(J, [0, 1]);
77         SE = circshift(J, [1, 1]);
78         S_ = circshift(J, [1, 0]);
79         SW = circshift(J, [1, -1]);
80         W = circshift(J, [0, -1]);
81         NW = circshift(J, [-1, -1]);
82
83         % Formowanie wektora sąsiadów
84         neighbors = [N(:) NE(:) E(:) SE(:) S_(:) SW(:) W(:) NW(:)];
85
86         % Liczenie warunków
87         % Przykład uproszczony - możesz zaimplementować bardziej dokładne odwzorowanie
            według tabelki
88         sum_neighbors = sum(neighbors, 2);
89         marker_vec = (sum_neighbors >= 2) & (sum_neighbors <= 6);
90         marker = reshape(marker_vec, size(J));
91
92         % Aktualizacja
93         J_old = J;
94         J(marker & J) = 0;
95         done = isequal(J, J_old);
96     end
97 end
98
99 J_cwsi = cwsi_skeletonization(I);
100
101 %% 6. Wyświetlenie wyników
102 figure('Name', 'Porównanie wyników', 'Units', 'normalized', 'OuterPosition', [0 0 1 1]);
103
104 subplot(2,2,1);
105 imshow(I); title('Oryginał - thintest.bmp');
106
107 subplot(2,2,2);
108 imshow(J_standard); title('Ścienianie - parametry standardowe');
109

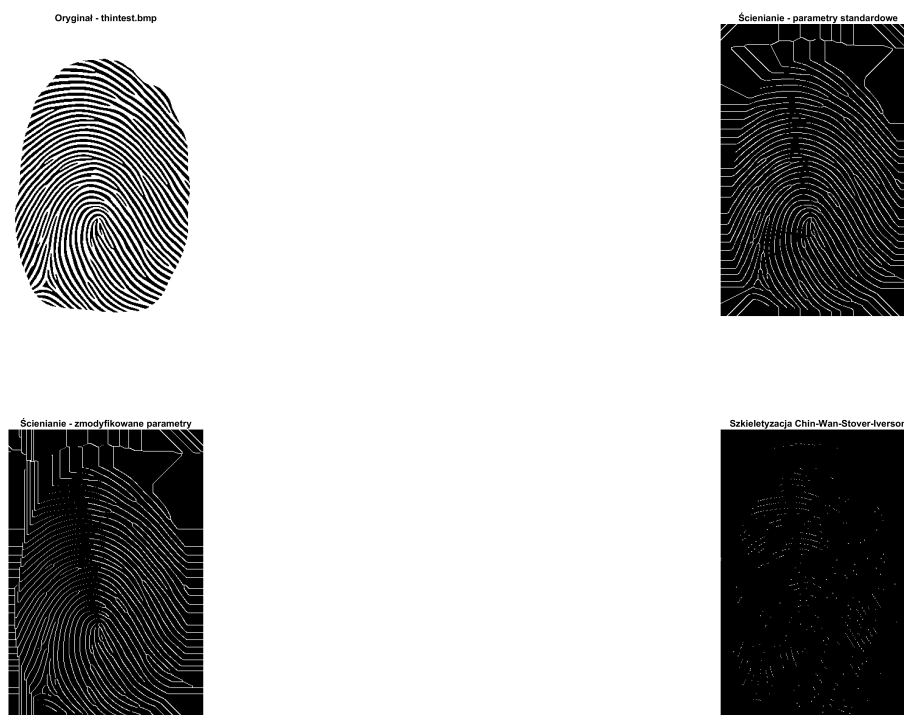
```

```

110 subplot(2,2,3);
111 imshow(J_modified); title('Ścienianie - zmodyfikowane parametry');
112
113 subplot(2,2,4);
114 imshow(J_cwsi); title('Szkieletyzacja Chin-Wan-Stover-Iverson');
115
116 % --- Save figure with high quality ---
117 exportgraphics(gcf, 'results.png', 'Resolution', 300);
118
119 %% 7. Dyskusja wyników
120 %
121 % W algorytmie standardowego ścieniania obraz zostaje redukowany warstwa po warstwie,
122 % a przy zmodyfikowanych parametrach proces jest bardziej agresywny.
123 %
124 % W przypadku szkieletyzacji Chin-Wan-Stover-Iverson widać szybsze uproszczenie struktur
125 % co skutkuje cienkimi, pojedynczymi liniami, bez fragmentacji dużych obszarów.
126 %
127 % Zmienność szybkości działania algorytmu wynika z faktu, że w początkowych iteracjach
128 % usuwanych jest więcej pikseli, a w końcowych tylko pojedyncze piksele - co powoduje
129 % zmniejszenie czasu każdej kolejnej iteracji.

```

3.3.1.3. Obrazy



Rys. 3.1. Porównanie wyników: Oryginał, Ścienianie standardowe, Ścienianie zmodyfikowane, Szkieletyzacja CWSI.

3.4. Wnioski

W przeprowadzonym ćwiczeniu dokonano analizy działania dwóch metod ścieniania obrazów binarnych:

- Metody klasycznej ścieniania z wykorzystaniem standardowych oraz zmodyfikowanych masek.

- Metody szkieletyzacji Chin-Wan-Stover-Iverson.

Na podstawie uzyskanych wyników można wyciągnąć następujące wnioski:

1. **Ścienianie z maskami standardowymi** pozwala na sukcesywne usuwanie zbędnych pikseli z obrazu, prowadząc do uzyskania cieńszych struktur. Proces ten jest stosunkowo stabilny, zachowując kształt linii papilarnych.
2. **Zmodyfikowane parametry ścieniania** powodują bardziej agresywne usuwanie pikseli. Widoczne jest zwiększone uproszczenie obrazu, jednak miejscami może to prowadzić do zniekształcenia struktury oryginalnych linii.
3. **Szkieletyzacja metodą Chin-Wan-Stover-Iverson** skutkuje szybkim i skutecznym redukowaniem obrazu do jego podstawowych konturów. Uzyskany szkielet jest bardzo cienki, jednak często prowadzi do fragmentacji obrazu, zwłaszcza w miejscach o większej gęstości pikseli.
4. **Szybkość działania algorytmu** jest największa na początku, gdy usuwane są większe grupy pikseli. W późniejszych iteracjach proces znacznie zwalnia, ponieważ zmiany obejmują pojedyncze piksele.
5. Obie metody pozwalają na skuteczne ścienianie obrazów binarnych, jednak wybór odpowiedniej metody oraz parametrów powinien być dostosowany do specyfiki przetwarzanego obrazu oraz wymagań aplikacji końcowej (np. analizy odcisków palców, rozpoznawania wzorców itp.).

Dzięki zastosowaniu różnych wariantów masek oraz metod możliwe jest dostosowanie procesu ścieniania do indywidualnych potrzeb, co stanowi istotny aspekt w cyfrowym przetwarzaniu obrazów.

4. Laboratorium 4

Biometryczne Systemy Zabezpieczeń	
Ćwiczenia Laboratoryjne	
Temat	TRANSFORMATA HOUGH
Ćwiczenie nr:	04
Autorzy:	Oleksii Nawrocki
Grupa laboratoryjna	02
Zespół nr.	XX
Data wykonania ćwiczenia	27.04.2025
Data oddania ćwiczenia	13.05.2025

4.1. Cel ćwiczenia

Celem zajęć jest zapoznanie się z transformacją Hough'a w zadaniu wykrywania linii i okręgów w testowych obrazach binarnych oraz obrazach tonowanych pozyskiwanych ze skanerów tęczówki oka.

4.2. Wstęp teoretyczny

4.3. Opis wykorzystywanego oprogramowania i narzędzi

4.4. Zadania do wykonania samodzielnego

4.4.1. Zadanie X

4.4.1.1. Treść

4.4.1.2. Przykładowy program

4.4.1.3. Obrazy

4.5. Wnioski

Na podstawie przeprowadzonych eksperymentów można stwierdzić, że:

Bibliografia

- [1] Matlab image processing toolbox documentation, 2024. Accessed: 2025-02-15.
- [2] Rafael Gonzalez and Richard Woods. *Digital Image Processing Global Edition*. Pearson Deutschland, 2017.

Spis rysunków

1.1	Wizualizacja obrazów w różnych formatach i kanałach.	10
1.2	Wizualizacja rozciągania i wyrównywania obrazów	12
1.3	Operacje segmentowanie progowaniem oraz otsu.	14
1.4	Detekcja obiektów na obrazie.	16
1.5	Operacje segmentowanie wieloprogowego.	18
2.1	Wizualizacja obrazów w różnych filtracjach splotowych.	25
3.1	Porównanie wyników: Oryginał, Ścienianie standardowe, Ścienianie zmodyfikowane, Szkieletyzacja CWSI.	32

Spis tabel

Spis listingów

1.1	Wczytanie obrazu i stworzenie konwersji	8
1.2	Wczytanie obrazu i przetwarzanie obrazu histogramami	11
1.3	Wczytanie obrazu i segmentacja na podstawie histogramu	13
1.4	Wczytanie obrazu i dokonanie detekcji obiektów na obrazie	15
1.5	Wczytanie obrazu i dokonanie detekcji obiektów na obrazie	17
2.1	Wczytanie obrazu i stworzenie konwersji	22
3.1	Wczytanie obrazu i stworzenie konwersji	30

Załącznik nr 2 do Zarządzenia nr 228/2021 Rektora Uniwersytetu Rzeszowskiego z dnia 1 grudnia 2021 roku w sprawie ustalenia procedury antyplagiatowej w Uniwersytecie Rzeszowskim

OŚWIADCZENIE STUDENTA O SAMODZIELNOŚCI PRACY

.....Oleksii Nawrocki.....

Imię (imiona) i nazwisko studenta

Wydział Nauk Ścisłych i Technicznych

.....Informatyka.....

Nazwa kierunku

.....oa131400.....

Numer albumu

1. Oświadczam, że moja praca projektowa pt.: Sprawozdanie z ćwiczeń laboratoryjnych z przedmiotu Biometryczne Systemy Zabezpieczeń
 - 1) została przygotowana przeze mnie samodzielnie*,
 - 2) nie narusza praw autorskich w rozumieniu ustawy z dnia 4 lutego 1994 roku o prawie autorskim i prawach pokrewnych (t.j. Dz.U. z 2021 r., poz. 1062) oraz dóbr osobistych chronionych prawem cywilnym,
 - 3) nie zawiera danych i informacji, które uzyskałem/am w sposób niedozwolony,
 - 4) nie była podstawą otrzymania oceny z innego przedmiotu na uczelni wyższej ani mnie, ani innej osobie.
2. Jednocześnie wyrażam zgodę/nie wyrażam zgody** na udostępnienie mojej pracy projektowej do celów naukowo-badawczych z poszanowaniem przepisów ustawy o prawie autorskim i prawach pokrewnych.

(miejscowość, data)

(czytelny podpis/y studenta/ów)

* Uwzględniając merytoryczny wkład prowadzącego przedmiot

** – niepotrzebne skreślić